Framework Selection Report

Pangasinan Heritage Digital Showcase

Course: Elective 4 - Special Topics in IT

Student: [STUDENT NAME - replace before submission]

Date: 31 August 2026

Selected option: Option A - React with Next.js 14 App Router

Alternative: Option B - Vue with Nuxt.js 3

---

1. Framework Decision

I selected React with Next.js 14 App Router for the Pangasinan Heritage Digital Showcase. Both Next.js and Nuxt can build a responsive, component-based static website, but Next.js is the better fit for this project because it matches the implemented React component library, supports static generation for all heritage pages, and allows interactive features to be isolated in small client components.

The decision was measured using seven criteria required by the activity. Each framework received a score from 1 to 5, where 5 means excellent project fit. The weighted result is 93.0/100 for Next.js and 84.5/100 for Nuxt.js.

Project requirements considered

- A responsive editorial website for Pangasinan heritage
- A searchable archive containing 41 heritage records
- A statically generated detail page for every record
- Reusable components following Atomic Design
- Deployment as static files through GitHub Pages
- Good performance, accessibility, and maintainable TypeScript code

2. Quantitative Comparison

Criterion	Weight	Next.js 14	Weighted	Nuxt.js 3	Weighted
Bundle size and performance	20%	4.5	18.0	4.5	18.0
Developer velocity	15%	4.5	13.5	4.0	12.0
Ecosystem maturity	10%	5.0	10.0	4.5	9.0
Learning curve	10%	4.0	8.0	3.5	7.0
Component architecture	15%	4.5	13.5	4.5	13.5
Documentation and community	10%	5.0	10.0	4.5	9.0
Suitability for project requirements	20%	5.0	20.0	4.0	16.0
Total	100%		93.0		84.5

The weighted contribution is calculated as `(score / 5) x weight`.

Brief justification by criterion

Criterion	Reason for the score
Bundle size and performance	Both frameworks support static generation. The completed Next.js build reports 102 kB first-load JavaScript for both Home and Heritage.
Developer velocity	Next.js uses the same React and TypeScript approach as the implemented component library, so less translation and retraining are required.
Ecosystem maturity	React and Next.js have a large, mature ecosystem for accessibility, testing, deployment, and future integrations.
Learning curve	The project already follows the React component model. Next.js still requires learning Server and Client Components, so it does not receive a perfect score.
Component architecture	Both support Atomic Design well. Next.js fits the existing TSX atoms, molecules, organisms, sections, and typed props.
Documentation and community	Both are well documented, but Next.js documentation directly covers the App Router, static exports, and the features used here.
Project suitability	Next.js directly supports the route structure, static detail-page generation, metadata, interactive search, and GitHub Pages export required by this project.

3. Why Next.js 14 Is More Appropriate

Next.js is more appropriate for this showcase for four main reasons:

1. It matches the actual implementation. The project already uses reusable React and TypeScript components for buttons, images, cards, search, navigation, grids, related places, and page sections.

2. It supports the required static website. `output: "export"` produces static HTML, CSS, JavaScript, and images. `generateStaticParams()` creates a detail page for each of the 41 heritage records without requiring a server.

3. It keeps interaction focused. Most page content can be rendered statically, while only search, filters, navigation, carousel controls, and motion need client-side React.

4. It fits the deployment target. The project includes repository-aware paths and a GitHub Actions workflow for deployment to GitHub Pages.

Nuxt.js 3 is still a capable alternative. It supports Vue components and static prerendering and could produce a similar website. However, choosing Nuxt would require converting the existing React component architecture into Vue components without providing a clear advantage for this particular project.

4. Final Recommendation

I recommend Option A: React with Next.js 14 App Router. Its score of 93.0/100 is higher than Nuxt.js 3 at 84.5/100. The main reason is project fit: Next.js supports the existing React Atomic Design system, statically generates the heritage collection and detail routes, limits browser JavaScript to interactive features, and exports the site for GitHub Pages.

The selection does not mean Nuxt.js is a poor framework. Next.js is selected because it is the more practical and coherent choice for the Pangasinan Heritage Digital Showcase that was actually designed and implemented.

References

- Next.js 14 App Router: https://nextjs.org/docs/14/app
- Next.js static export: https://nextjs.org/docs/14/app/building-your-application/deploying/static-exports
- Nuxt.js rendering: https://nuxt.com/docs/3.x/guide/concepts/rendering
- React components: https://react.dev/learn/describing-the-ui